scoping at the language level, so all constraints need to be explicitly stated in rule bodies.

- **Action**—Actions are also unconstrained; they are often just a string like "read", "write", or "deploy" within `input.action`. Rego treats this the same as any other data, so it's flexible but lacks structure. Matching is usually done with simple equality checks inside rules.
- **Resource**—Similar to principals, resources are arbitrary JSON objects; commonly under `input.resource`. Because Rego can work with deeply nested and dynamic data, it's powerful for evaluating real-world resource structures. However, this also means there's no assurance that the resource object conforms to a known type or shape, which can cause subtle bugs if keys are missing or misnamed.
- **Context**—Context is where Rego shines. Policies can take into account time, location, network conditions, request headers, or any other metadata provided through input or external data sources. This makes it especially valuable for runtime enforcement decisions that rely on environmental or organizational state.

Overall, Rego's approach to PARC is very flexible, but that flexibility has a cost. Without types, scopes, or language-level structures for principals, actions, or resources, policies need to be written carefully and tested thoroughly to ensure they are correct. This makes Rego a strong choice for infrastructure policy—especially in complex, data-rich environments—but it can be more demanding for teams that want to implement detailed authorization at scale.

8.2.4 Strengths and limitations

OPA and Rego gave ACME's DevOps team the tools they needed to secure modern infrastructure. Policies were portable, testable, and independent of the applications they protected. However, as the use of OPA expanded, so did concerns about complexity. Rego's logic programming model has a steep learning curve, and it's easy to write policies that are correct but difficult to read or maintain. The absence of structure or types means validation, analysis, and testing are critical, but the language itself does not support them.